

TransitOps

Consolidated Metrics, Assignment Algorithm & KPI Formula Reference

This document defines: (1) the small set of high-level metrics to show as summary cards at the top of every dashboard, (2) the vehicle/driver assignment algorithm used when a trip is created, and (3) the exact formula, data source, and recommended chart type for every KPI across the four role dashboards. Source is marked **Manual** (typed in by a user), **Derived** (computed purely from other stored fields), or **Hybrid** (a computed value that depends on at least one manually entered field, such as acquisition cost or revenue).

1. Consolidated High-Level Metrics (Top Summary Cards)

These six metrics form a global “overview strip” that can sit at the top of every dashboard (values filtered/scoped by role where relevant), so every user sees fleet health at a glance before drilling into their own page. Order reflects the sequence they should appear left to right.

KPI	Formula / Implementation	Source	Best Visualization
1. Active trips	COUNT(Trip) WHERE status = 'DISPATCHED'	Derived	Big number card, with a small trend sparkline vs. yesterday
2. Fleet utilization %	COUNT(Vehicle WHERE status='ON_TRIP') / COUNT(Vehicle WHERE status != 'RETIRED') × 100	Derived	Radial gauge or donut, 0–100%
3. Compliance rate %	COUNT(Driver WHERE status != 'SUSPENDED' AND licenseExpiryDate > today) / COUNT(Driver WHERE status != 'SUSPENDED') × 100	Derived	Donut with red segment for non-compliant, or a single colored badge (green/amber/red)
4. Net profit (period)	SUM(Trip.revenue) – (SUM(FuelLog.cost) + SUM(MaintenanceLog.cost) + SUM(Expense.amount)) for the selected period	Hybrid	Big number card, colored red/green by sign, with a 7/30-day trend line beneath
5. Vehicles in maintenance	COUNT(Vehicle) WHERE status = 'IN_SHOP'	Derived	Big number card with a small icon badge; click-through to Maintenance Log
6. Open safety flags	COUNT(Driver WHERE licenseExpiryDate < today+30) + COUNT(Driver WHERE status='SUSPENDED') + COUNT(open Incident)	Derived	Big number card, amber/red badge if > 0

Implementation note: compute these six with a single lightweight aggregation query (or scheduled materialized view refreshed every 1–5 min) rather than recomputing on every page load per-role — all four dashboards read from the same summary object, just render different subsets.

2. Trip → Vehicle & Driver Assignment Algorithm

When a Fleet Manager creates a trip, the system should not just show every Available vehicle/driver and let them pick blindly — it should filter out invalid options (hard rules) and then rank the valid remainder (soft scoring) so the top suggestion is pre-selected.

2.1 Hard filters (must pass, or excluded from the list entirely)

- `Vehicle.status == AVAILABLE` (excludes On Trip, In Shop, Retired)
- `Vehicle.maxLoadCapacity ≥ Trip.cargoWeight`
- `Driver.status == AVAILABLE` (excludes On Trip, Off Duty, Suspended)
- `Driver.licenseExpiryDate > today`
- `Driver.licenseCategory` is compatible with `Vehicle.type` (e.g. heavy-goods category required for trucks)

2.2 Soft scoring (rank the vehicles/drivers that pass the filters)

Score each valid (vehicle, driver) pair and suggest the highest score as the default, while still letting the Fleet Manager override manually.

Factor	Formula (normalized 0–1)	Suggested weight	Why it matters
Capacity fit	$1 - \frac{ \text{Vehicle.maxLoadCapacity} - \text{Trip.cargoWeight} }{\text{Vehicle.maxLoadCapacity}}$	0.30	Rewards using a right-sized vehicle instead of always grabbing the biggest truck available
Driver safety score	<code>Driver.safetyScore / 100</code> (normalized 0–1)	0.25	Prefers safer drivers for dispatch when multiple are eligible
Maintenance risk	$1 - \min(1, (\text{Vehicle.odometer} - \text{lastServiceOdometer}) / \text{serviceIntervalKm})$	0.20	Avoids assigning a vehicle that's near its next scheduled service
Fair rotation	$1 - (\text{Driver.tripsToday} / \text{max tripsToday across all eligible drivers})$	0.15	Distributes trips evenly instead of always picking the same driver
Idle time bonus	<code>min(1, hoursSinceLastTrip / 24)</code> for the vehicle	0.10	Slightly favors vehicles that have been sitting idle longest, improving utilization

$\text{TotalScore} = 0.30 \times \text{CapacityFit} + 0.25 \times \text{SafetyScore} + 0.20 \times \text{MaintenanceRisk} + 0.15 \times \text{FairRotation} + 0.10 \times \text{IdleBonus}$.
Weights are a starting point — tune them once real usage data is available; for the hackathon, hardcoding these five weights is fine.

2.3 Pseudocode

```
function suggestAssignment(trip):
  candidateVehicles = Vehicle.filter(status = AVAILABLE
                                     AND maxLoadCapacity >= trip.cargoWeight)
  candidateDrivers  = Driver.filter(status = AVAILABLE
                                    AND licenseExpiryDate > today
                                    AND licenseCategory compatible with trip.vehicleType)

  if candidateVehicles is empty or candidateDrivers is empty:
    return NO_MATCH # surface "no eligible vehicle/driver" to Fleet Manager
  best = null
```

```
for v in candidateVehicles:
    for d in candidateDrivers:
        score = 0.30 * capacityFit(v, trip)
            + 0.25 * (d.safetyScore / 100)
            + 0.20 * maintenanceRisk(v)
            + 0.15 * fairRotation(d)
            + 0.10 * idleBonus(v)
        if best is null or score > best.score:
            best = { vehicle: v, driver: d, score: score }
```

```
return best # pre-fill the dispatch form; Fleet Manager can still override
```

This runs client-side or server-side at trip-creation time only (not continuously), since it's triggered by a single user action rather than needing to be a background job.

3. KPI Formula Reference by Role

Exact computation, whether the underlying value is typed in by a user or derived purely from other records, and the recommended chart type for each KPI referenced in the dashboard designs.

3.1 Fleet Manager

KPI	Formula / Implementation	Source	Best Visualization
Fleet utilization %	COUNT(Vehicle WHERE status='ON_TRIP') / COUNT(Vehicle WHERE status != 'RETIRED') × 100	Derived	Radial gauge, updates live
Vehicle status breakdown	GROUP BY Vehicle.status, COUNT(*)	Derived	Donut chart, one segment per status
Vehicles in maintenance + ETA	COUNT(Vehicle WHERE status='IN_SHOP'); ETA = MaintenanceLog.expectedCompletionDate	Hybrid	Table/list; ETA field is manually entered when the record is created
Upcoming preventive maintenance	nextServiceOdometer = lastServiceOdometer + serviceIntervalKm (config); flag when Vehicle.odometer ≥ nextServiceOdometer – threshold	Hybrid	serviceIntervalKm is a manually configured setting; timeline/calendar view of upcoming due dates
Idle vehicle alert	daysSinceLastTrip = today – MAX(Trip.completedAt) WHERE vehicleId=X; flag if status=AVAILABLE AND daysSinceLastTrip > N	Derived	List with a badge/chip, sortable by idle days
Vehicle document expiry	daysToExpiry = DocumentExpiryDate – today	Manual	DocumentExpiryDate is typed in; table color-coded green/amber/red by days remaining
Average vehicle age / odometer	AVG(today – Vehicle.acquisitionDate); AVG(Vehicle.odometer)	Hybrid	acquisitionDate is manual; shown as a single trend card or small bar chart by vehicle type

3.2 Driver

KPI	Formula / Implementation	Source	Best Visualization
My current trip status	Trip.status WHERE driverId = currentUser AND status IN (DRAFT, DISPATCHED)	Derived	Status card/badge (Draft/Dispatched)
License days to expiry	Driver.licenseExpiryDate – today	Hybrid	licenseExpiryDate is manual (Safety Officer entry); shown as a countdown badge
My safety score	Set by Safety Officer manually, or SafetyScore = weighted avg of trip incident penalties + telematics if integrated	Manual	Radial gauge 0–100, read-only

KPI	Formula / Implementation	Source	Best Visualization
My trip history	Trip.filter(driverId=currentUser, status=COMPLETED) ORDER BY completedAt DESC	Derived	Simple table list
My vehicle condition	Vehicle.status WHERE id = trip.vehicleId	Derived	Status badge

3.3 Safety Officer

KPI	Formula / Implementation	Source	Best Visualization
Compliance rate %	COUNT(Driver WHERE status != 'SUSPENDED' AND licenseExpiryDate > today) / COUNT(Driver WHERE status != 'SUSPENDED') × 100	Derived	Single donut/gauge, green ≥90%, amber 75–90%, red <75%
Expiring licenses (30/60/90d)	COUNT(Driver) WHERE licenseExpiryDate BETWEEN today AND today+30 (and +60, +90 buckets)	Derived	Stacked bar, one bar per bucket, or a color-coded table
Suspended drivers	COUNT(Driver WHERE status='SUSPENDED')	Derived	Big number card + list with suspension reason (manual field) and date
Safety score distribution	HISTOGRAM(Driver.safetyScore) in bins of 10	Manual	safetyScore itself is manually set/updated; visualized as a histogram or leaderboard table sorted ascending
Incident count per driver	COUNT(Incident) WHERE driverId = X AND status='OPEN'	Manual	Incident records are manually logged; shown as a badge next to driver name in the directory

3.4 Financial Analyst

KPI	Formula / Implementation	Source	Best Visualization
Net profit	$\text{SUM}(\text{Trip.revenue}) - (\text{SUM}(\text{FuelLog.cost}) + \text{SUM}(\text{MaintenanceLog.cost}) + \text{SUM}(\text{Expense.amount}))$, scoped to a date range and optionally a vehicle	Hybrid	Big number card (green if positive, red if negative) + line chart over time
Vehicle ROI	$(\text{Revenue} - (\text{Maintenance} + \text{Fuel})) / \text{Vehicle.acquisitionCost}$, per vehicle	Hybrid	Horizontal bar chart, vehicles ranked best to worst; acquisitionCost and revenue are manually entered inputs
Operational cost breakdown	$\text{SUM}(\text{FuelLog.cost})$, $\text{SUM}(\text{MaintenanceLog.cost})$, $\text{SUM}(\text{Expense.amount})$ grouped by category	Derived	Stacked bar or donut, one segment per cost category
Fuel efficiency	$\text{SUM}(\text{Trip.actualDistance}) / \text{SUM}(\text{FuelLog.liters})$, per vehicle	Derived	Bar chart per vehicle, or line chart trend over time
Cost per km	$\text{TotalOperationalCost} / \text{SUM}(\text{Trip.actualDistance})$, per vehicle or fleet-wide	Derived	Single KPI card, or bar chart per vehicle for comparison
Revenue by route/region	$\text{SUM}(\text{Trip.revenue})$ GROUP BY (source, destination) or region	Hybrid	revenue is manually entered per trip; visualized as a bar chart or simple map if lat/long is captured
Expense trend	$\text{SUM}(\text{Expense.amount})$ GROUP BY week/month, by category	Manual	Expense entries are manual; shown as a stacked area or line chart over time

Legend: **Manual** = typed in directly by a user (no computation). **Derived** = computed entirely from other stored records via a query/aggregation, no manual input required. **Hybrid** = a computed formula that consumes at least one manually entered field (e.g. acquisitionCost, revenue, licenseExpiryDate, expected completion date).